GENERAL STORE MANAGEMENT SOFTWARE

Python Project Report

Submitted by: Shubh Agrawal
B.Tech First Year
VIT Bhopal University

Academic Project — VITyarthi: Build Your Own Project

Academic Year 2026–27

1. Introduction

This project is a command-line based General Store Management Software developed in Python. The main idea is to keep the basic billing process simple while also maintaining a small amount of sales information during a program session.

The application uses a fixed list of common general-store products with their prices. A store user can start a bill, select products using product codes, add multiple products to one transaction and complete the bill. The program also records how many times each product has been selected and stores the totals of completed bills.

The project was developed as an academic application to demonstrate practical use of basic Python concepts such as lists, functions, loops, conditional statements, user input and program control flow.

2. Problem Statement

In a small store, even simple billing requires repeated addition and keeping track of products sold. A basic computerized system can reduce this manual work by calculating the total automatically and keeping a session-wise record of sales.

The problem addressed by this project is therefore to create a simple Python program that can perform basic store billing and provide a history of product quantities sold and completed bill totals.

3. Objectives

• Create a simple menu-driven store management application.

• Allow the user to select products using product codes.

• Calculate the total amount of a bill automatically.

• Allow multiple products to be added to a single bill.

• Maintain product-wise sales counts during the current session.

• Maintain the totals of completed bills during the current session.

• Apply basic input validation for menu choices and product codes.

• Use functions and Python data structures to keep the program understandable.

4. Functional Requirements

4.1 Main Menu

The main menu acts as the control point of the application. It provides options to start billing, check history and exit.

4.2 Billing

The billing function displays the product list with codes and prices. After the user enters a valid product code, the corresponding price is added to the running total and the

product's sales count is increased. The user can continue adding products or checkout.

4.3 Sales History

The history function displays the quantity sold for each product and the total amount of each completed bill stored during the current program run.

4.4 Input Validation

The program checks whether menu selections and product codes are within the expected numeric ranges. Invalid numeric choices produce a message asking the user to enter a valid operation.

5. Non-Functional Requirements

Requirement	How the project addresses it
Usability	Simple numbered menu and clear console messages.
Performance	Small in-memory lists make operations immediate for the intended scale.
Reliability	Menu and billing loops continue until the user chooses the relevant exit/checkout option.
Maintainability	Product data, billing logic and history logic are separated into lists and functions.
Error handling	Invalid numeric menu/product choices are handled with validation messages.
Resource efficiency	Uses basic Python data structures and no external libraries.

6. System Architecture

The application follows a simple procedural architecture. The user interacts with the main menu, which routes the request either to the billing function or the history function. Product names, prices, sales counts and bill totals are maintained in Python lists while the program is running.

System Workflow / Architecture

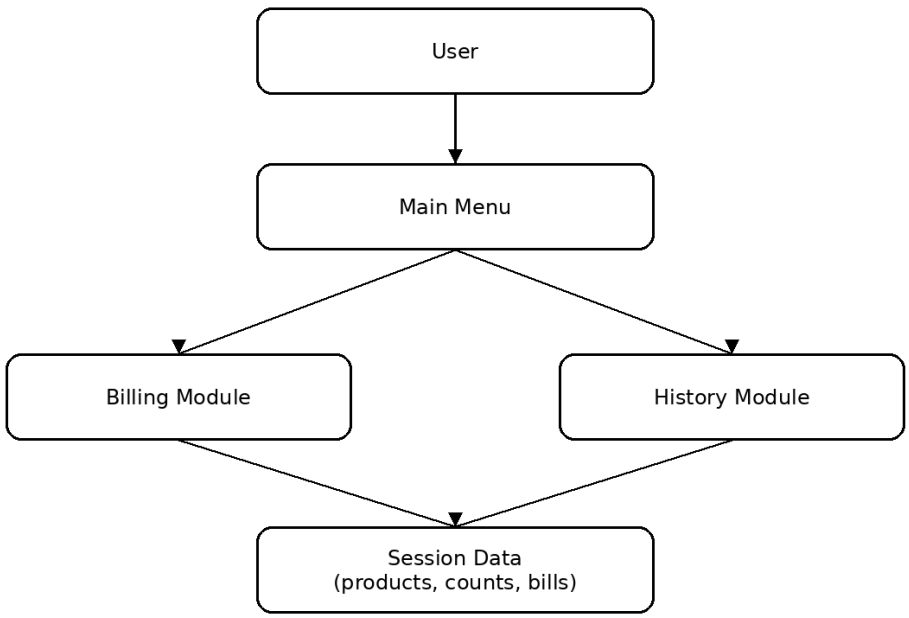


Figure 1. High-level workflow and data flow of the application.

7. Design Diagrams

7.1 Use Case Diagram

The main actor is the store user. The user can start billing, add products, checkout, view history and exit the application.

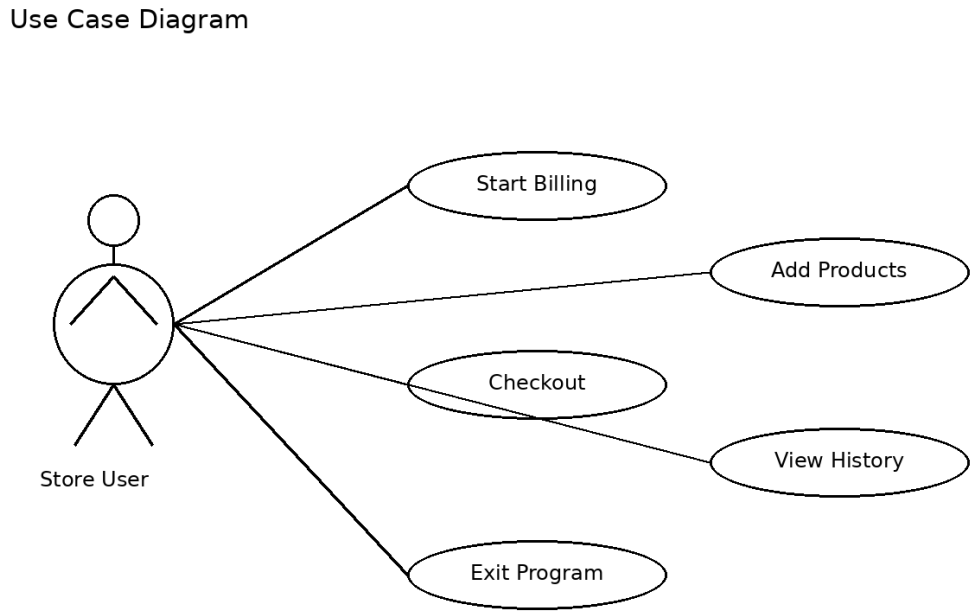


Figure 2. Use case diagram.

7.2 Workflow Diagram

The workflow begins at the main menu. Selecting billing opens the product-selection loop. After checkout, the bill total is stored and control returns to the main menu. Selecting history displays the session data. The program ends when the user chooses exit.

7.3 Sequence Diagram

A typical billing sequence is: Store User → Main Menu → Billing Function → Product Selection → Running Total and Sales Count Update → Checkout → Bill Stored → Main Menu. This is the sequence implemented by the current procedural code.

7.4 Class / Component Diagram

The current project does not use Python classes. Its components are implemented procedurally through the main menu, the billing function, the history function and the shared lists containing product and session data. A class-based design is a possible future improvement.

7.5 ER / Database Design

A database is not used in the current version, so an ER diagram is not applicable. The equivalent session data is represented by the products, prices, counts and bills lists in

memory.

8. Design Decisions and Rationale

Lists were selected because the project contains a small fixed set of products and each product can be accessed by its numeric position. This keeps the implementation straightforward for a first-year Python project.

The billing operation is placed inside a function so that the main program does not contain all of the transaction logic. The history operation is also separated into its own function. This gives the program a clearer structure while keeping the implementation easy to understand.

A command-line interface was selected because the main objective is to demonstrate programming logic rather than graphical interface development. It also keeps the project lightweight and requires no external packages.

9. Implementation Details

9.1 Product Data

The program stores nine products and their prices in two lists. A separate counts list records how many times each product has been selected.

9.2 Billing Logic

The billing function starts the total at zero. It displays the available products, accepts a product code, adds the corresponding price, updates the product count and asks whether the user wants to continue or checkout.

9.3 Bill History

After checkout, the final total is appended to the bills list. The history function later prints these totals together with the product-wise sales counts.

9.4 Control Flow

The main menu runs inside a loop and continues until the user enters the exit option. Conditional statements determine which operation is performed.

10. Screenshots / Results

The following screenshots show representative output generated from the submitted program. For the final college submission, it is preferable to replace these with screenshots captured directly on the student's own computer while running the final version.

Main Menu

```
Welcome to the Management Software of our Store
Menu
Enter 0 to start billing
Enter 1 to check history
Enter 2 to exit
EnterThankyou
```

Figure 3. Main menu and exit operation.

Billing and Checkout

```
Welcome to the Management Software of our Store
Menu
Enter 0 to start billing
Enter 1 to check history
Enter 2 to exit
EnterCode Products   Prices

1    Milk       70
2    Cheese     150
3    Butter     60
4    Chocolate  20
5    Juice      30
6    Chips      10
7    Biscuits   10
8    Maggie     15
9    Coffee     50
Enter Product Code for inputMilk Added
Total amount = 70
Enter 0 to add more product
Enter 1 to checkout
EnterComplete Payment
Final total is 70
Menu
Enter 0 to start billing
Enter 1 to check history
Enter 2 to exit
EnterThankyou
```

Figure 4. Product selection and billing output.

Billing followed by History

```
Welcome to the Management Software of our Store
Menu
Enter 0 to start billing
Enter 1 to check history
Enter 2 to exit
EnterCode Products   Prices

1    Milk       70
2    Cheese     150
3    Butter     60
4    Chocolate  20
5    Juice      30
6    Chips      10
7    Biscuits   10
8    Maggie     15
9    Coffee     50
Enter Product Code for inputMilk Added
Total amount = 70
Enter 0 to add more product
Enter 1 to checkout
EnterComplete Payment
Final total is 70
Menu
Enter 0 to start billing
Enter 1 to check history
Enter 2 to exit
EnterProducts   Quantity Sold

Milk       1
Cheese     0
Butter     0
Chocolate  0
Juice      0
Chips      0
Biscuits   0
Maggie     0
Coffee     0

Bill 1 : 70

Menu
Enter 0 to start billing
Enter 1 to check history
Enter 2 to exit
EnterThankyou
```

Figure 5. Billing/history output.

11. Testing Approach

Testing was carried out using normal user flows and invalid numeric inputs. The main purpose was to check that the menu routes the user correctly, product selection changes the running total, checkout stores the bill and history displays the stored information.

Test	Input / Action	Expected Result
T1	Select 2 from main menu	Program exits.
T2	Select 0 and enter a valid product code	Product is added and total increases.
T3	Add two products to one bill	Both prices are included in the final total.
T4	Enter an invalid product code	Program displays a valid-code message.

T5	Enter an invalid menu option	Program displays a valid-operation message.
T6	Checkout and then open history	Completed bill and product counts are displayed.

One known limitation is that the current implementation converts user input directly with int(). Therefore, entering a non-numeric value such as a letter will raise a Python ValueError. This is documented rather than hidden because improving robust input handling is a clear future enhancement.

12. Challenges Faced

• Designing the billing loop so that the user can add multiple products before checkout.

• Keeping the running bill total separate from the product-wise sales counts.

• Maintaining bill history for multiple completed transactions during one execution.

• Handling invalid numeric choices without unnecessarily terminating the main program.

• Keeping the interface simple enough to understand while still providing the required store operations.

13. Learnings and Key Takeaways

• Learned how lists can be used to store and process related data.

• Understood how functions can divide a program into separate logical operations.

• Practised while loops for menu-driven applications.

• Used conditional statements to control different user choices.

• Learned how user input can be processed to perform a real-world task.

• Understood the importance of validation and testing in an interactive program.

• Learned that even a small project benefits from separating data, processing and user interaction.

14. Future Enhancements

• Store products, bills and sales history permanently using a file or database.

• Add stock quantity and low-stock alerts.

• Allow the user to enter quantities directly.

• Generate a formatted bill containing individual items and quantities.

• Add customer information and transaction numbers.

• Improve input validation using try/except.

• Create a graphical user interface.

• Split the project into multiple Python modules and add automated tests.

15. References

1. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines, provided for the course project.

2. Python 3 concepts used in the implementation: lists, functions, loops, conditional statements and standard input/output.

16. Submission Checklist

• README.md included in the GitHub repository.

• statement.md included in the GitHub repository.

• Final Python source code included.

• Project report exported as PDF for portal submission.

• System architecture / workflow diagram included.

• Use case diagram included.

• Testing approach and results included.

• Project screenshots included; replace sample screenshots with your own final-run screenshots if possible.

• GitHub repository checked to ensure the repository link opens and contains the complete project files.